

One loop, one measured limit

Checksum-verified OpenMP scaling of an independent M2
simulation engine

COSC3500 — Milestone 2

September 2026

**Question: how far does one OpenMP optimisation scale before
serial CSR rebuilding dominates?**

See the workload first



Blue prey flee red hunters. The clip shows 256 agents; the benchmark uses 100,000.

`docs/presentation/demo.mp4`: illustrative engine capture, not benchmark evidence

The claim is deliberately narrow

Engine boundary

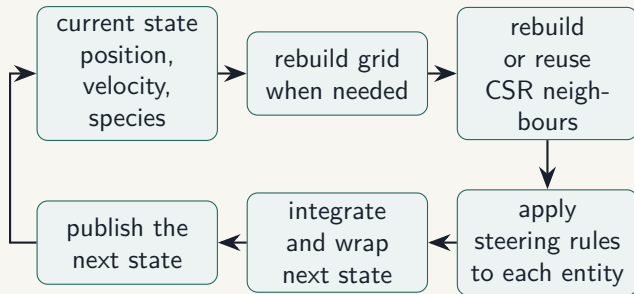
- ▶ Deterministic fixed-step state engine
- ▶ 2D simulation; renderer supplies 2.5D presentation
- ▶ Scenario selects a kernel and rules
- ▶ Renderer is outside the timed loop

Measured M2 workload

- ▶ Continuous predator–prey kernel
- ▶ 100,000 agents, 10 fixed steps, radius 80
- ▶ Seeds 31, 32 and 33
- ▶ One allocated Rangpur virtual CPU node

I am not claiming every scene or kernel has the same speed-up.

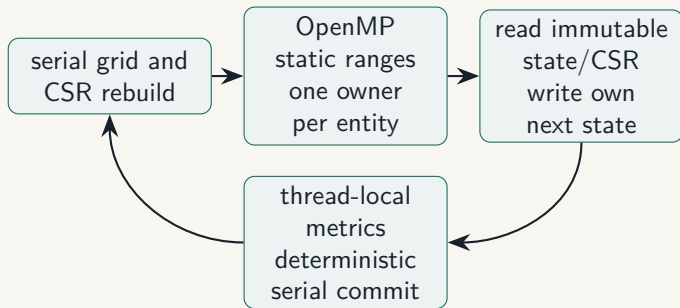
One step, then repeat



Naive discovery: $\Theta(n^2)$ candidate tests **Grid + CSR:** $\Theta(n + m)$, where m is local candidate work

After the final step, hash the state; rendering stays outside the timed loop.

The optimisation: parallelise only safe per-entity reuse



Invariant

- ▶ Disjoint entity ownership
- ▶ Immutable current state and CSR
- ▶ Same seed, rule order and checksum oracle

What changes

- ▶ L7 reuse loop becomes OpenMP work-sharing
- ▶ Grid/CSR rebuild remains serial

How we measured



Control	Value
Samples	1 warm-up + 5 timed samples per tuple
Inputs	3 seeds; checksum must match per seed
Placement	one vcpu-5 node; CPUs 0-7; one NUMA node
Build	GCC 13.3.1; recorded optimisation and OpenMP flags
Metric	median ns/entity-update; retain per-seed spread

Only checksum-matched runs enter the charts.

The measured comparison and its missing baseline

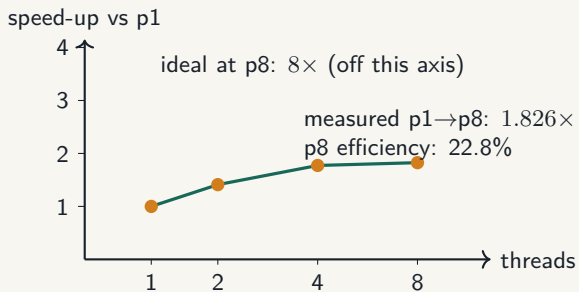
Variant	Engine	Threads	ns/entity-update	Ratio
L0 baseline	M1	1	358.239	1.000×
L7 serial	M2	1	94.945	3.773×
L7 OpenMP	M2	1	95.185	3.759×
L7 OpenMP	M2	8	52.131	6.863×

6.863× crosses from M1 to M2; it cannot measure M2 optimisation alone.

A same-codebase M2 L0 → M2 L7 run is prepared. No M2 optimisation ratio is claimed yet. The parallel comparison starts at OpenMP p1.

Median of seed medians; 3 seeds × 5 timed samples, workload 100K × 10 steps

Measured strong scaling: useful gain, early plateau



Threads	1	2	4	8
Median ns/entity-update	95.185	67.600	53.846	52.131
Speed-up vs p1	1.000	1.410	1.773	1.826
Efficiency	100.0%	70.5%	44.3%	22.8%

Ratios are matched per seed then medianed; raw rows retain the seed-level values

Fixed-density scaling: speed-up holds, per-entity cost rises

Agents	p1 ns/unit	p8 ns/unit	S_8	η_8	seed range at p8
100K	94.684	51.700	1.831	22.9%	51.522–52.261
200K	101.307	53.689	1.885	23.6%	53.006–53.982
400K	110.450	59.458	1.855	23.2%	59.449–59.952
800K	120.254	65.394	1.836	22.9%	65.335–66.287

Fixed density

World area grows proportionally with N ; each run uses 10 fixed steps. S_8 stays near $1.8\times$ across 100K–800K.

Weak-scaling check

100K p1 $\rightarrow$ 800K p8: normalized elapsed time is $5.520\times$; weak-scaling efficiency is 18.1%.

Per-entity time rises with N at both p1 and p8. This is observed behaviour; no hardware cause is asserted because counters were unavailable.

3 seeds $\times$ 5 timed samples; all serial/OpenMP checksum gates passed

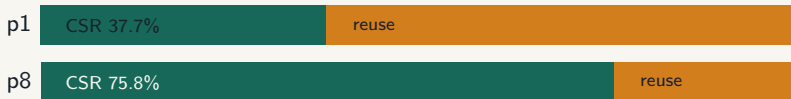
Why scaling plateaus: instrumented phase evidence

At p8, median diagnostic time

- ▶ CSR rebuild: about 33.5 ms (75.8%)
- ▶ CSR reuse/publish: about 10.6 ms (24.0%)
- ▶ The parallel loop shrinks; rebuild does not

Inside the serial rebuild

- ▶ grid/count traversal: about 14.0 ms (42%)
- ▶ second fill traversal: about 10.8 ms (32%)
- ▶ residual allocation/prefix work: about 8.7 ms (26%)



This is an instrumented diagnostic build: its milliseconds explain the shape but are not comparable to release timings.

Decision: optimise the rebuild next, but only after an A/B gate

Evidence-supported next candidate

- ▶ Avoid the second CSR fill traversal when a bounded scratch edge list can safely retain accepted pairs
- ▶ Preserve pair order and checksum
- ▶ Fall back to the existing traversal when the bounded scratch space is full

Do not claim it yet

- ▶ The CSR candidate A/B has no validated result here
- ▶ Promote only with interleaved, checksum-matched p1/p2/p4/p8 runs
- ▶ Reject a gain that is not stable across all three seeds

CUDA, MPI and hand-written AVX remain unmeasured for this workload.

Validity, limits and reproducibility

What this establishes

- ▶ Independent M2 executable, not M1 reuse
- ▶ Exact final state per seed across serial and thread counts
- ▶ Repeated one-node strong-scaling evidence
- ▶ A measured explanation for the p8 plateau

What it does not establish

- ▶ All scenes, densities or initial conditions
- ▶ Multi-node, GPU or SIMD speed-up
- ▶ Hardware-counter attribution: perf stat was unavailable on this allocation
- ▶ A speed-up for the unvalidated CSR candidate

Raw data: `results/profile/run-20260924T110021Z-rG8ANs`; phase data:
`results/profile/phase-615532`

Conclusion

1. Static ownership preserved each seed's checksum and gave $1.826\times$ at 8 threads
2. At p8, serial CSR rebuild occupied 75.8% of instrumented time; more workers offer little until that changes
3. Test the bounded scratch candidate against the original binary; reject it if memory or p1 cost outweighs p8 benefit

AI disclosure

Generative AI assisted implementation, data checks and slide drafting. I will independently review the final evidence and defend every claim in the interview.

Next test: the bounded CSR candidate against the original M2 binary.